

FROM A BLANK COMPUTER TO A RUNNING LARAVEL APP

Install Mobiliti UG step by step.

A visual onboarding guide for Windows and macOS—covering VS Code, Git over HTTPS, Laragon or ServBay, DBeaver, PHP, Node.js, Composer, database setup, and the first local run.

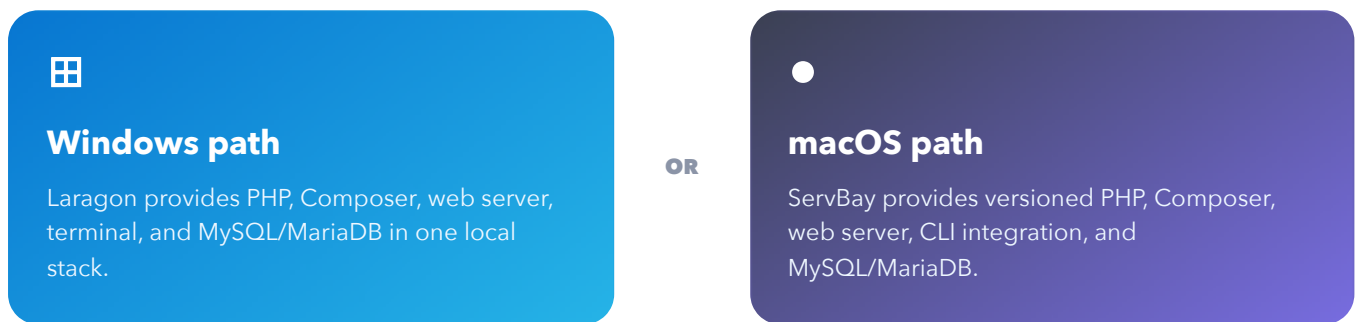
**Laravel 10****PHP 8.2****Node 16.13.0****MySQL / MariaDB****Beginner friendly**

Prepared for UiTM Mobiliti UG · Based on repository commit **3a9ddbb59d** · 11 August 2026

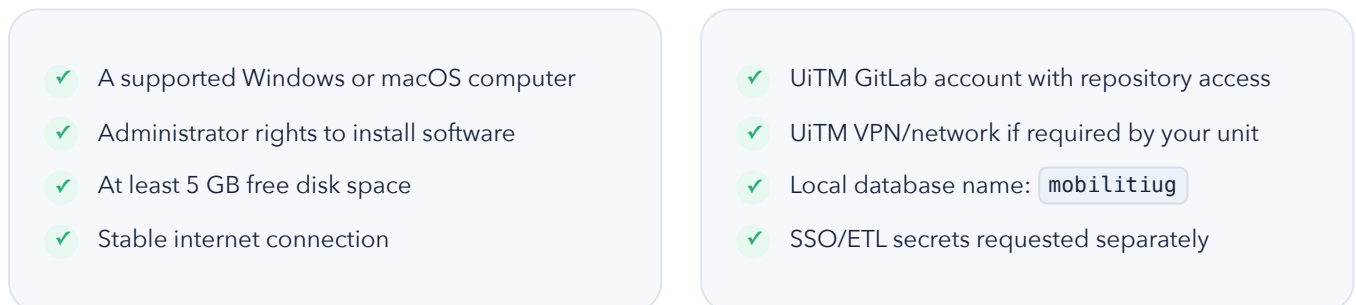
YOUR ROUTE IN ONE GLANCE

What you are building

You will create a complete local development workspace. Pick one operating-system path; the Git, DBeaver, project, and Laravel steps are shared.



Before you begin



! **Version rule:** this repository requires PHP **8.2** and its README pins Node.js **16.13.0** / **NPM 8.14.0**. Node 16 is end-of-life, so use it only for this legacy project and never as your default for new work.

FROM THE REPOSITORY README

Know the project before you install it

Mobility UG began as a UiTM web core that gathers the common foundations needed by application modules into one Laravel system.

Purpose

Provide reusable building blocks so new modules can share the same access, navigation, role, and reference-data foundations.

Core capabilities

- ✓ Assign users
- ✓ Configure menus
- ✓ Configure roles and permissions
- ✓ Manage parameter and lookup data

Technology foundation

LAYER	README SPECIFICATION	HOW IT IS USED HERE
Application	Laravel 10.x · PHP 8.2	Backend framework and runtime
Frontend	Vuexy · Bootstrap 5.1 · Sass · jQuery 3.6	Interface, styling, and browser behaviour
Build	Node 16.13.0 · NPM 8.14.0	Current scripts compile assets with Laravel Mix
Data	MySQL / MariaDB	Application, audit, ETL, and integration connections

! **Build-tool clarification:** the README describes Gulp as part of the Vuexy toolset; the repository's current `package.json` runs Laravel Mix through `npm run dev`.

Key package families named in the README

HTTP + identity

Guzzle
Socialite
Laravel Impersonate

Access + navigation

Laravel Permission
Laravel Menu
Route Attributes

Data presentation

Laravel DataTables
Server-side tables and application listings

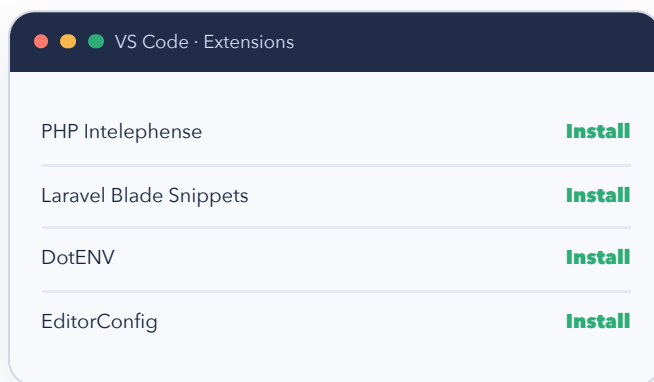
→ The final page turns the README's controller, secured-view, parameter-setting, and theme notes into a compact developer reference.

STEP 1

Install your editor and Git

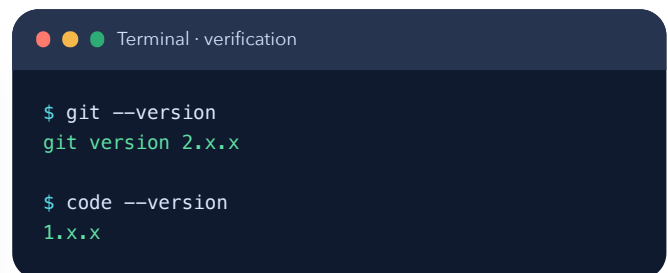
1 Visual Studio Code

1. Open **code.visualstudio.com/download**.
2. Download the stable installer for your OS.
3. **Windows:** select "Add to PATH" and "Open with Code" during setup.
4. **macOS:** drag VS Code to Applications; in Command Palette run **Shell Command: Install 'code' command in PATH**.



2 Git

1. Open **git-scm.com/downloads**.
2. **Windows:** run Git for Windows; keep recommended defaults and VS Code as editor.
3. **macOS:** run `xcode-select --install`, or use the installer linked by Git.
4. Close and reopen the terminal after installation.



Recommended Git identity

```
$ git config --global user.name "Your Name"
$ git config --global user.email "your.name@uitm.edu.my"
$ git config --global --list
```

i Use your own identity. These values appear in future commits. They are not your GitLab password and do not grant repository access.

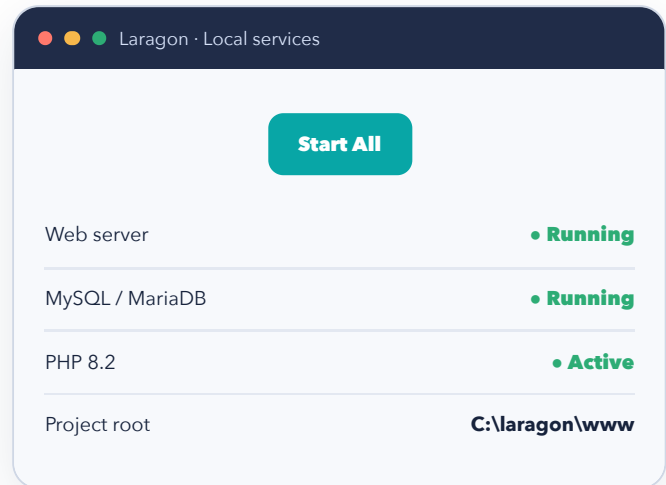
Official downloads: <https://code.visualstudio.com/download> · <https://git-scm.com/downloads>

STEP 2A

Set up Laragon on Windows

1. Download Laragon from laragon.org/download.
2. Install to the default `C:\laragon` location.
3. Keep **Auto Virtual Hosts** enabled.
4. Launch Laragon and click **Start All**.
5. Confirm Apache/Nginx and MySQL/MariaDB show as running.

! If ports **80/443** or **3306** are already used, stop the other web/database stack before starting Laragon.



Select PHP 8.2

From Laragon: **Menu** → **PHP** → **Version** → **8.2.x**. If 8.2 is absent, use **Menu** → **Tools** → **Quick add** → **PHP 8.2** (availability varies), then restart all services.

```
$ php -v           # must show PHP 8.2.x
$ composer --version # Composer 2.x
$ php -m           # check required extensions
```

PHP extensions

Confirm: **curl, fileinfo, gd, intl, mbstring, openssl, pdo_mysql, sodium, xml, zip**.

Database

Use MySQL or MariaDB on **127.0.0.1:3306**. Root password is often blank on a fresh Laragon setup.

Web root

This guide clones into **C:\laragon\www\mobilitiug**. Laragon can create a **mobilitiug.test** host.

i **Node comes next.** Use NVM for Windows so this project can stay on Node 16.13.0 without forcing every other project to use an obsolete Node release.

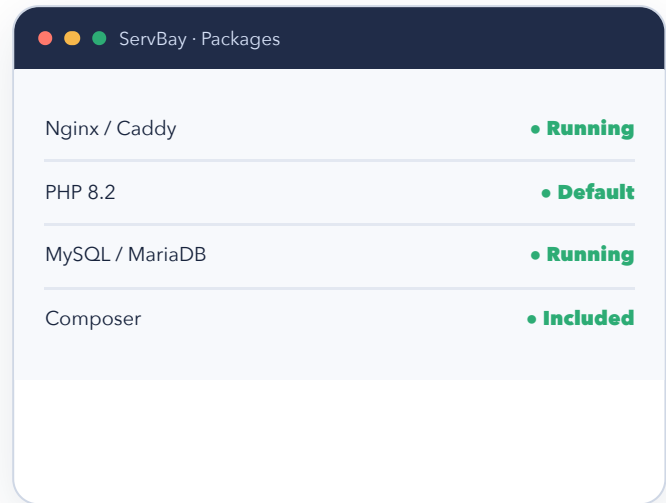
Official guide: <https://laragon.org/docs/install>

STEP 2B

Set up ServBay on macOS

1. Download the stable macOS build at **servbay.com/download**.
2. Open the DMG and drag ServBay to Applications.
3. Launch it and complete initialization.
4. In **Packages**, install/enable a web server, **PHP 8.2**, and MySQL or MariaDB.
5. Start the enabled services.

! Stop other local stacks if ports **80/443** or **3306** conflict. ServBay's default web root is **/Applications/ServBay/www**.



Enable command-line access

Open **ServBay** → **Settings** → **Command Line Tools**, install support for **zsh**, choose PHP 8.2 as the CLI default, then close and reopen Terminal.

```

$ php -v           # must show PHP 8.2.x
$ composer --version # included with ServBay
$ php -m           # confirm pdo_mysql, mbstring, xml, zip...

```

Add a site (optional)

After cloning, add a ServBay Site whose root points to **/Applications/ServBay/www/mobilitiug/public**, select PHP 8.2, and let ServBay create local HTTPS.

Simplest first run

You can skip site configuration initially and use **php artisan serve**. This isolates the Laravel setup from web-server configuration.

i **Composer is already included.** Do not install a second global Composer unless `composer --version` still fails after enabling ServBay's CLI tool.

Official guides: [ServBay installation](#) · [Using PHP](#) · [CLI support](#)

STEP 3

Clone Mobiliti UG over HTTPS

With your local stack ready, sign in to UiTM GitLab and confirm that the project opens. Then clone into the correct web root.

Windows • Laragon

Open **Menu** → **Laragon** → **Terminal**, then:

```
$ cd C:\laragon\www
$ git clone
https://git.uitm.edu.my/bsm/mobilitiug.git
$ cd mobilitiug
$ code .
```

macOS • ServBay

Open Terminal, then clone inside ServBay's web root:

```
$ cd /Applications/ServBay/www
$ git clone
https://git.uitm.edu.my/bsm/mobilitiug.git
$ cd mobilitiug
$ code .
```

What HTTPS authentication looks like

git.uitm.edu.my · Sign in / token

Username: your UiTM GitLab username

Password: personal access token (if required)

If your normal password is rejected:
GitLab may require a Personal Access Token for HTTPS.
Create one in GitLab account settings with only the repository scope your unit requires. Treat it like a password.

! Never paste credentials into the clone URL, source code, screenshots, or `.env`. Let Git Credential Manager / macOS Keychain store them securely when prompted.

Quick check

```
$ git branch --show-current
Development
$ git remote -v
origin https://git.uitm.edu.my/bsm/mobilitiug.git (fetch)
origin https://git.uitm.edu.my/bsm/mobilitiug.git (push)
$ git status
On branch Development
```

STEPS 4-5

Pin Node, then create the database

Windows • NVM for Windows

1. Download **nvm-setup.exe** from the official releases page.
2. Open a new **Administrator PowerShell**.

```
> nvm install 16.13.0
> nvm use 16.13.0
> node -v
v16.13.0
> npm -v
8.1.0
```

The repository says NPM 8.14.0. If exact parity is needed: `npm i -g npm@8.14.0`.

macOS • nvm

1. Install nvm using the current command in its official README.
2. Close/reopen Terminal, then:

```
$ nvm install 16.13.0
$ nvm use 16.13.0
$ node -v
v16.13.0
$ npm -v
8.1.0
```

Optional exact NPM parity: `npm i -g npm@8.14.0`.

Install DBeaver Community

1. Download DBeaver Community from **dbeaver.io/download**.
2. Create a new **MySQL** connection.
3. Host: **127.0.0.1**, Port: **3306**.
4. User: **root**; use your local stack's root password.
5. Click **Test Connection** and allow the driver download.

DBeaver - MySQL connection

Server Host · 127.0.0.1

Port · 3306

Username · root

Password ·

Test Connection

```
-- Open SQL Editor in DBeaver and run:
CREATE DATABASE mobilitiug
CHARACTER SET utf8mb4
COLLATE utf8mb4_unicode_ci;
```

Official sources: [NVM for Windows](#) · [nvm for macOS](#) · [Node 16.13.0](#) · [DBeaver](#)

STEP 6

Create a safe local `.env`

In the project folder, copy the example file. Never edit `.env.example` for personal credentials.

```
# Windows
> copy .env.example .env
```

```
# macOS
$ cp .env.example .env
```

Minimum local values

```
APP_NAME="Mobiliti UG"
APP_DEBUG=true
DB_CONNECTION=mysql
DB_PORT=3306
DB_USERNAME=root
TELESCOPE_ENABLED=false
AUDIT_DB_CONNECTION=mysql
CACHE_DRIVER=array
SESSION_DRIVER=file

APP_ENV=local
APP_URL=http://127.0.0.1:8000
DB_HOST=127.0.0.1
DB_DATABASE=mobilitiug
DB_PASSWORD=your_local_password
TELESCOPE_DB_CONNECTION=mysql
SCOUT_DRIVER=null
QUEUE_CONNECTION=sync
MAIL_MAILER=log
```

! **Why `SCOUT_DRIVER=null` ?** It prevents the first local run from contacting the shared Meilisearch server. Enable Meilisearch only after your team supplies a local/shared host and key.

UiTM-connected values: request, do not invent

FEATURE	VARIABLES	WHEN NEEDED
Staff/student ETL	DB_ETL_HOST, DB_ETL_DATABASE, DB_ETL_USERNAME, DB_ETL_PASSWORD	UiTM VPN/network
NILAMS integration	DB_NILAMS_HOST, DB_NILAMS_DATABASE, DB_NILAMS_USERNAME, DB_NILAMS_PASSWORD	Specific modules
UiTM SSO	OAuth_CLIENT_ID, OAuth_CLIENT_SECRET	Real sign-in
Search	SCOUT_DRIVER=meilisearch, MEILISEARCH_HOST, MEILISEARCH_KEY	Indexing/search

! **Do not copy values from deployment files or teammates' machines.** Obtain development credentials through the approved UiTM owner and keep the `.env` file out of Git.

STEPS 7-9

Install packages, migrate, launch

Run these commands from the repository root—the folder that contains `artisan`, `composer.json`, and `package.json`.

```
Terminal · mobilitiug

$ php -v                # PHP 8.2.x
$ node -v               # v16.13.0
$ composer install
$ npm install
$ php artisan key:generate
$ php artisan optimize:clear
$ php artisan migrate
Migration table created successfully.
```

! **Private Composer packages:** `composer install` may request access to UiTM GitLab packages. Authenticate using your approved GitLab method. Do not commit Composer auth files or tokens.

Run in two terminals

A

Laravel server

```
$ php artisan serve
```

Keep this terminal open. Default URL: **http://127.0.0.1:8000**.

B

Frontend assets

```
$ npm run dev
```

Compiles Laravel Mix assets. Keep it open if BrowserSync remains active; use `npm run watch` during ongoing frontend work.

Optional UiTM-connected commands

Run only after ETL + Meilisearch access works:

```
$ php artisan scout:sync-index-settings
$ php -d memory_limit=-1 artisan scout:import "App\Models\UserMgmt\UserStaff"
$ php -d memory_limit=-1 artisan scout:import "App\Models\UserMgmt\UserStudent"
$ php -d memory_limit=-1 artisan scout:import "App\Models\UserMgmt\Department"
$ php artisan auth:create-admin-user <staff_id_uitm>
$ php artisan migrate:system-setting --sync-meilisearch
```

✓ **First checkpoint:** the browser should reach Laravel without a PHP fatal error. A failed SSO/ETL-dependent screen is an access/configuration issue, not necessarily a failed local installation.

FINAL CHECKPOINT

Confirm success—and fix the common blockers

You're ready.

Open the local URL, then confirm the app renders, migrations completed, and both build terminals stay free of fatal errors.

- ✓ PHP 8.2 active
- ✓ Node 16.13.0 active
- ✓ Database connected
- ✓ Laravel key generated
- ✓ Frontend compiled

http://127.0.0.1:8000

Mobility UG

Local app responding

Troubleshooting map

"php" / "composer" not found

Enable Laragon/ServBay CLI integration, reopen the terminal, then verify PATH.

`php -v` `composer -V`

PHP version conflict

Select PHP 8.2 in the stack, reopen terminal, then clear Laravel caches.

`php artisan optimize:clear`

SQLSTATE connection refused

Start MySQL/MariaDB; verify host 127.0.0.1, port 3306, credentials, and database name.

`php artisan migrate:status`

Git clone denied / 2FA error

Confirm project membership and VPN. Use an approved HTTPS token when GitLab requires it.

`git remote -v`

Composer package denied

Your project access may not include the private UiTM packages. Ask the repository owner for package access.

`composer install -vvv`

Node / Sass build errors

Re-select Node 16.13.0, verify NPM, then reinstall dependencies through the normal project workflow.

`node -v` `npm -v`

Port already in use

Stop the duplicate web/database service, or run Laravel on another port.

`php artisan serve --port=8001`

Search / SSO / ETL fails

These need approved UiTM endpoints, secrets, and possibly VPN. Keep Scout null for an offline first run.

`SCOUT_DRIVER=null`

Useful daily commands

Update: `git pull` · `composer install` after Composer lock changes · `npm install` after NPM lock changes · `php artisan migrate` after migrations · `php artisan optimize:clear` after environment/config changes.

Guide basis: repository README, composer.json, composer.lock, package.json, package-lock.json, .env.example, and config files at commit 3a9ddb59d. Product details checked against official documentation on 11 August 2026. Generated cover artwork used the built-in image generation workflow.

CONTINUE AFTER INSTALLATION

Build modules the project way

These conventions come from the repository README. Use them after the local application is running and the required UiTM integrations are available.

Create a resource controller

```
$ php artisan make:controller-core \
  <Module Name>/<Role Name>/\
  <Controller Name> --resource
```

1. Set the module URL prefix.
2. Set the route name and permission.
3. Synchronize the system settings when the search connection is ready.

Create a secured view

Add the new Blade file under **resources/views/secured** and begin with:

```
@extends('layouts/secured')

@section('content')
  <!-- Module content -->
@endsection
```

Keep module views within the secured layout so navigation, identity, and permissions stay consistent.

Parameter settings and lookup data

Values for one category

```
use App\FacadeService\
  ParameterSettingService;

ParameterSettingService::
  parameter('<name_category>')
  ->pluck('name', 'id');
```

Category with parameters

```
use App\FacadeService\
  ParameterSettingService;

ParameterSettingService::
  categoryParameter(
    '<name_category>')->get();
```

Theme and navigation colours

Edit **resources/assets/scss/variables/_variables.scss**:

```
$primary: #E65100;
$success: #bbc032;
$warning: #bf360c;
$dark: #231c10;

$secondary: #fb8c00;
$info: #ffc107;
$danger: #d84315;
// then rebuild assets
```

! **Connected command:** run `php artisan migrate:system-setting --sync-meilisearch` only after Meilisearch access is configured. It is not part of the offline first run.

README references: [Laravel Permission documentation](#) · [Laravel Route Attributes](#) · [project controller generator](#) · [secured Blade layout](#) · [ParameterSettingService](#) · [Vuexy SCSS variables](#).